

15.2.13.2 JOIN Clause

MySQL supports the following JOIN syntax for the ***table_references*** part of SELECT statements and multiple-table DELETE and UPDATE statements:

```

table_references:
    escaped_table_reference [, escaped_table_reference] ...

escaped_table_reference: {
    table_reference
  | { 0J table_reference }
}

table_reference: {
    table_factor
  | joined_table
}

table_factor: {
    tbl_name [PARTITION (partition_names)]
      [[AS] alias] [index_hint_list]
  | [LATERAL] table_subquery [AS] alias [(col_list)]
  | ( table_references )
}

joined_table: {
    table_reference {[INNER | CROSS] JOIN | STRAIGHT_JOIN} table_factor [join_specification]
  | table_reference {LEFT|RIGHT} [OUTER] JOIN table_reference join_specification
  | table_reference NATURAL [INNER | {LEFT|RIGHT} [OUTER]] JOIN table_factor
}

join_specification: {
    ON search_condition
  | USING (join_column_list)
}

join_column_list:
    column_name [, column_name] ...

index_hint_list:
    index_hint [ index_hint] ...

index_hint: {
    USE {INDEX|KEY}
      [FOR {JOIN|ORDER BY|GROUP BY}] ([index_list])

```

```

| {IGNORE|FORCE} {INDEX|KEY}
  [FOR {JOIN|ORDER BY|GROUP BY}] (index_list)
}

index_list:
  index_name [, index_name] ...

```

A table reference is also known as a join expression.

A table reference (when it refers to a partitioned table) may contain a `PARTITION` clause, including a list of comma-separated partitions, subpartitions, or both. This option follows the name of the table and precedes any alias declaration. The effect of this option is that rows are selected only from the listed partitions or subpartitions. Any partitions or subpartitions not named in the list are ignored. For more information and examples, see Section 26.5, “Partition Selection”.

The syntax of ***table_factor*** is extended in MySQL in comparison with standard SQL. The standard accepts only ***table_reference***, not a list of them inside a pair of parentheses.

This is a conservative extension if each comma in a list of ***table_reference*** items is considered as equivalent to an inner join. For example:

```

SELECT * FROM t1 LEFT JOIN (t2, t3, t4)
      ON (t2.a = t1.a AND t3.b = t1.b AND t4.c = t1.c)

```

is equivalent to:

```

SELECT * FROM t1 LEFT JOIN (t2 CROSS JOIN t3 CROSS JOIN t4)
      ON (t2.a = t1.a AND t3.b = t1.b AND t4.c = t1.c)

```

In MySQL, `JOIN`, `CROSS JOIN`, and `INNER JOIN` are syntactic equivalents (they can replace each other). In standard SQL, they are not equivalent. `INNER JOIN` is used with an `ON` clause, `CROSS JOIN` is used otherwise.

In general, parentheses can be ignored in join expressions containing only inner join operations. MySQL also supports nested joins. See Section 10.2.1.8, “Nested Join Optimization”.

Index hints can be specified to affect how the MySQL optimizer makes use of indexes. For more information, see Section 10.9.4, “Index Hints”. Optimizer hints and the `optimizer_switch` system variable are other ways to influence optimizer use of indexes. See Section 10.9.3, “Optimizer Hints”, and Section 10.9.2, “Switchable Optimizations”.

The following list describes general factors to take into account when writing joins:

- A table reference can be aliased using ***tbl_name AS alias_name*** or ***tbl_name alias_name***:

```
SELECT t1.name, t2.salary
  FROM employee AS t1 INNER JOIN info AS t2 ON t1.name = t2.name;

SELECT t1.name, t2.salary
  FROM employee t1 INNER JOIN info t2 ON t1.name = t2.name;
```

- A ***table_subquery*** is also known as a derived table or subquery in the FROM clause. See Section 15.2.15.8, “Derived Tables”. Such subqueries *must* include an alias to give the subquery result a table name, and may optionally include a list of table column names in parentheses. A trivial example follows:

```
SELECT * FROM (SELECT 1, 2, 3) AS t1;
```

- The maximum number of tables that can be referenced in a single join is 61. This includes a join handled by merging derived tables and views in the FROM clause into the outer query block (see Section 10.2.2.4, “Optimizing Derived Tables, View References, and Common Table Expressions with Merging or Materialization”).
- **INNER JOIN** and **,** (comma) are semantically equivalent in the absence of a join condition: both produce a Cartesian product between the specified tables (that is, each and every row in the first table is joined to each and every row in the second table).

However, the precedence of the comma operator is less than that of **INNER JOIN**, **CROSS JOIN**, **LEFT JOIN**, and so on. If you mix comma joins with the other join types when there is a join condition, an error of the form Unknown column '**col_name**' in 'on clause' may occur. Information about dealing with this problem is given later in this section.

- The ***search_condition*** used with **ON** is any conditional expression of the form that can be used in a **WHERE** clause. Generally, the **ON** clause serves for conditions that specify how to join tables, and the **WHERE** clause restricts which rows to include in the result set.
- If there is no matching row for the right table in the **ON** or **USING** part in a **LEFT JOIN**, a row with all columns set to **NULL** is used for the right table. You can use this fact to find rows in a table that have no counterpart in another table:

```
SELECT left_tbl.*
FROM left_tbl LEFT JOIN right_tbl ON left_tbl.id = right_tbl.id
WHERE right_tbl.id IS NULL;
```

This example finds all rows in `left_tbl` with an `id` value that is not present in `right_tbl` (that is, all rows in `left_tbl` with no corresponding row in `right_tbl`). See Section 10.2.1.9, “Outer Join Optimization”.

- The `USING(join_column_list)` clause names a list of columns that must exist in both tables. If tables `a` and `b` both contain columns `c1`, `c2`, and `c3`, the following join compares corresponding columns from the two tables:

```
a LEFT JOIN b USING (c1, c2, c3)
```

- The `NATURAL [LEFT] JOIN` of two tables is defined to be semantically equivalent to an `INNER JOIN` or a `LEFT JOIN` with a `USING` clause that names all columns that exist in both tables.
- `RIGHT JOIN` works analogously to `LEFT JOIN`. To keep code portable across databases, it is recommended that you use `LEFT JOIN` instead of `RIGHT JOIN`.
- The `{ OJ ... }` syntax shown in the join syntax description exists only for compatibility with ODBC. The curly braces in the syntax should be written literally; they are not metasyntax as used elsewhere in syntax descriptions.

```
SELECT left_tbl.*
FROM { OJ left_tbl LEFT OUTER JOIN right_tbl
      ON left_tbl.id = right_tbl.id }
WHERE right_tbl.id IS NULL;
```

You can use other types of joins within `{ OJ ... }`, such as `INNER JOIN` or `RIGHT OUTER JOIN`. This helps with compatibility with some third-party applications, but is not official ODBC syntax.

- `STRAIGHT_JOIN` is similar to `JOIN`, except that the left table is always read before the right table. This can be used for those (few) cases for which the join optimizer processes the tables in a suboptimal order.

Some join examples:

```

SELECT * FROM table1, table2;

SELECT * FROM table1 INNER JOIN table2 ON table1.id = table2.id;

SELECT * FROM table1 LEFT JOIN table2 ON table1.id = table2.id;

SELECT * FROM table1 LEFT JOIN table2 USING (id);

SELECT * FROM table1 LEFT JOIN table2 ON table1.id = table2.id
      LEFT JOIN table3 ON table2.id = table3.id;

```

Natural joins and joins with USING, including outer join variants, are processed according to the SQL:2003 standard:

- Redundant columns of a NATURAL join do not appear. Consider this set of statements:

```

CREATE TABLE t1 (i INT, j INT);
CREATE TABLE t2 (k INT, j INT);
INSERT INTO t1 VALUES(1, 1);
INSERT INTO t2 VALUES(1, 1);
SELECT * FROM t1 NATURAL JOIN t2;
SELECT * FROM t1 JOIN t2 USING (j);

```

In the first SELECT statement, column `j` appears in both tables and thus becomes a join column, so, according to standard SQL, it should appear only once in the output, not twice. Similarly, in the second SELECT statement, column `j` is named in the USING clause and should appear only once in the output, not twice.

Thus, the statements produce this output:

```

+-----+-----+-----+
| j     | i     | k     |
+-----+-----+-----+
| 1     | 1     | 1     |
+-----+-----+-----+
| j     | i     | k     |
+-----+-----+-----+
| 1     | 1     | 1     |
+-----+-----+-----+

```

Redundant column elimination and column ordering occurs according to standard SQL, producing this display order:

- First, coalesced common columns of the two joined tables, in the order in which they occur in the first table
- Second, columns unique to the first table, in order in which they occur in that table
- Third, columns unique to the second table, in order in which they occur in that table

The single result column that replaces two common columns is defined using the coalesce operation. That is, for two `t1.a` and `t2.a` the resulting single join column `a` is defined as `a = COALESCE(t1.a, t2.a)`, where:

```
COALESCE(x, y) = (CASE WHEN x IS NOT NULL THEN x ELSE y END)
```

If the join operation is any other join, the result columns of the join consist of the concatenation of all columns of the joined tables.

A consequence of the definition of coalesced columns is that, for outer joins, the coalesced column contains the value of the non-NULL column if one of the two columns is always NULL. If neither or both columns are NULL, both common columns have the same value, so it doesn't matter which one is chosen as the value of the coalesced column. A simple way to interpret this is to consider that a coalesced column of an outer join is represented by the common column of the inner table of a JOIN. Suppose that the tables `t1(a, b)` and `t2(a, c)` have the following contents:

```
t1      t2
----  ----
1 x    2 z
2 y    3 w
```

Then, for this join, column `a` contains the values of `t1.a`:

```
mysql> SELECT * FROM t1 NATURAL LEFT JOIN t2;
+-----+-----+-----+
| a     | b     | c     |
+-----+-----+-----+
| 1     | x     | NULL  |
```

	2	y	z

By contrast, for this join, column a contains the values of t2.a.

```
mysql> SELECT * FROM t1 NATURAL RIGHT JOIN t2;
+-----+-----+-----+
| a      | c      | b      |
+-----+-----+-----+
|      2 | z      | y      |
|      3 | w      | NULL   |
+-----+-----+-----+
```

Compare those results to the otherwise equivalent queries with JOIN ... ON:

```
mysql> SELECT * FROM t1 LEFT JOIN t2 ON (t1.a = t2.a);
+-----+-----+-----+-----+
| a      | b      | a      | c      |
+-----+-----+-----+-----+
|      1 | x      | NULL   | NULL   |
|      2 | y      |      2 | z      |
+-----+-----+-----+-----+
```

```
mysql> SELECT * FROM t1 RIGHT JOIN t2 ON (t1.a = t2.a);
+-----+-----+-----+-----+
| a      | b      | a      | c      |
+-----+-----+-----+-----+
|      2 | y      |      2 | z      |
| NULL  | NULL   |      3 | w      |
+-----+-----+-----+-----+
```

- A USING clause can be rewritten as an ON clause that compares corresponding columns. However, although USING and ON are similar, they are not quite the same. Consider the following two queries:

```
a LEFT JOIN b USING (c1, c2, c3)
a LEFT JOIN b ON a.c1 = b.c1 AND a.c2 = b.c2 AND a.c3 = b.c3
```

With respect to determining which rows satisfy the join condition, both joins are semantically identical.

With respect to determining which columns to display for `SELECT *` expansion, the two joins are not semantically identical. The `USING` join selects the coalesced value of corresponding columns, whereas the `ON` join selects all columns from all tables. For the `USING` join, `SELECT *` selects these values:

```
COALESCE(a.c1, b.c1), COALESCE(a.c2, b.c2), COALESCE(a.c3, b.c3)
```

For the `ON` join, `SELECT *` selects these values:

```
a.c1, a.c2, a.c3, b.c1, b.c2, b.c3
```

With an inner join, `COALESCE(a.c1, b.c1)` is the same as either `a.c1` or `b.c1` because both columns have the same value. With an outer join (such as `LEFT JOIN`), one of the two columns can be `NULL`. That column is omitted from the result.

- An `ON` clause can refer only to its operands.

Example:

```
CREATE TABLE t1 (i1 INT);
CREATE TABLE t2 (i2 INT);
CREATE TABLE t3 (i3 INT);
SELECT * FROM t1 JOIN t2 ON (i1 = i3) JOIN t3;
```

The statement fails with an `Unknown column 'i3' in 'on clause'` error because `i3` is a column in `t3`, which is not an operand of the `ON` clause. To enable the join to be processed, rewrite the statement as follows:

```
SELECT * FROM t1 JOIN t2 JOIN t3 ON (i1 = i3);
```

- `JOIN` has higher precedence than the comma operator (`,`), so the join expression `t1, t2 JOIN t3` is interpreted as `(t1, (t2 JOIN t3))`, not as `((t1, t2) JOIN t3)`. This affects statements that use an `ON` clause because that clause can refer only to columns in the operands of the join, and the precedence affects interpretation of what those operands are.

Example:

```
CREATE TABLE t1 (i1 INT, j1 INT);
CREATE TABLE t2 (i2 INT, j2 INT);
CREATE TABLE t3 (i3 INT, j3 INT);
INSERT INTO t1 VALUES(1, 1);
INSERT INTO t2 VALUES(1, 1);
INSERT INTO t3 VALUES(1, 1);
SELECT * FROM t1, t2 JOIN t3 ON (t1.i1 = t3.i3);
```

The JOIN takes precedence over the comma operator, so the operands for the ON clause are t2 and t3. Because t1.i1 is not a column in either of the operands, the result is an Unknown column 't1.i1' in 'on clause' error.

To enable the join to be processed, use either of these strategies:

- Group the first two tables explicitly with parentheses so that the operands for the ON clause are (t1, t2) and t3:

```
SELECT * FROM (t1, t2) JOIN t3 ON (t1.i1 = t3.i3);
```

- Avoid the use of the comma operator and use JOIN instead:

```
SELECT * FROM t1 JOIN t2 JOIN t3 ON (t1.i1 = t3.i3);
```

The same precedence interpretation also applies to statements that mix the comma operator with INNER JOIN, CROSS JOIN, LEFT JOIN, and RIGHT JOIN, all of which have higher precedence than the comma operator.

- A MySQL extension compared to the SQL:2003 standard is that MySQL permits you to qualify the common (coalesced) columns of NATURAL or USING joins, whereas the standard disallows that.